Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Register Spill Behaviour and Cache Hit/Miss Rates on RISC-V SPEC CPU2017

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Ismail Akturk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

4 November 2026

gem5 TimingSimpleCPU · workload profile `test` · L1D 4 KiB, L1I 32 KiB, L2 32 KiB, L1D 2-way, L2 8-way · gem5 `e0a12aa4e3+dirty`

Definitions (metric schema 9). Every stack store issued inside the region of interest, and every later load that reads it back (the load's address lies in the bytes the store wrote; widths may differ), belongs to exactly one class. `spill`: register-allocator spill code, identified by GCC itself (`-dP` RTL dumps of the same build, memory operands in the LRA spill area). `csave`: callee-saved register save/restore, return address included, identified from the binary's CFI (`.eh_frame`). `other_stack`: every remaining SP/FP-relative stack store and reload (stack-passed arguments, memory-resident locals). `stack_*` is the sum of the three. For each class an *event* is one (store, first reload) pair, so `event` $\leq$ `store` and `load` $\geq$ `event`. All counters cover only the region delimited by `m5_work_begin/m5_work_end`: the gem5 statistics are dumped and reset at those boundaries, so the cache columns span the same window, and the cache tags carry the class, so per-class hit/miss rates come straight from gem5. *Spill %* and *Csave %* are the class's store+reload instructions over `ROI_insts`; *spill/stack %* is the share of stack events that are real spills.

Benchmark	ROI_insts	stack_event	**spill_event**	csave_event	other_event	spill/stack %	Spill %	Csave %	L1D m %	L2 m %	sp_load m %	sp_store m %	cs_load m %	cs_store m %
cactuBSSN	48 023 254 960	3 820 802 773	3 797 752 332	22 956 427	94 014	99.4	21.32	0.10	15.764	64.32	4.453	3.914	0.126	0.159
parest	37 262 401 281	130 109 038	16 148 624	110 058 554	3 901 860	12.4	0.22	0.59	11.595	57.67	9.578	3.513	2.118	1.649
deepsjeng	46 202 686 169	1 453 401 396	380 604 124	1 046 384 737	26 412 535	26.2	2.78	4.53	9.980	6.89	2.859	7.613	3.282	3.898
leela	26 063 891 644	808 702 134	26 356 444	749 189 404	33 156 286	3.3	0.36	5.75	6.901	20.72	9.698	0.425	1.766	0.859
namd	31 914 751 099	336 702 115	334 454 030	1 939 340	308 745	99.3	3.13	0.01	12.887	25.71	5.486	1.313	3.844	3.891
lbm	6 506 800 040	274 131 044	274 130 500	544	0	100.0	9.67	0.00	23.735	88.75	0.424	2.261	15.074	0.184
nab	6 691 483 051	152 456 167	124 535 928	24 348 560	3 571 679	81.7	7.90	0.73	6.914	4.63	1.389	2.123	0.898	1.211
xz	2 014 351 521	34 435 119	4 683 669	26 392 608	3 358 842	13.6	0.68	2.62	6.496	25.90	0.124	1.002	2.222	2.241
xz_4m_l1	2 001 462 938	30 569 523	3 959 703	23 548 312	3 061 508	13.0	0.59	2.35	7.996	35.15	0.121	0.808	2.863	1.815
xz_4m_l2	2 083 103 943	29 428 092	3 700 891	22 776 791	2 950 410	12.6	0.54	2.19	9.797	42.07	0.116	0.814	3.396	1.908
xz_4m_l3e	10 573 516 663	134 644 921	61 575 948	66 069 008	6 999 965	45.7	2.51	1.25	6.916	19.19	1.914	1.167	1.747	3.242
xz_4m_l4	4 011 967 585	68 464 242	20 213 033	44 411 548	3 839 661	29.5	2.12	2.21	8.369	30.25	2.255	2.394	1.558	2.249
xz_4m_l4e	12 115 711 352	137 799 962	63 036 496	67 532 844	7 230 622	45.7	2.28	1.11	6.702	17.71	2.020	1.170	1.810	3.475
xz_1m_l0	711 271 503	13 362 560	1 091 347	10 938 543	1 332 670	8.2	0.62	3.08	5.939	29.24	0.168	2.012	1.021	1.382
xz_1m_l1	719 848 327	12 739 426	973 549	10 458 508	1 307 369	7.6	0.58	2.91	6.727	33.78	0.168	1.706	1.215	1.441
xz_1m_l2	724 690 222	12 498 311	956 442	10 241 945	1 299 924	7.7	0.57	2.83	6.928	34.23	0.148	1.718	1.254	1.484
xz_1m_l3e	1 994 406 562	34 952 797	12 876 553	20 010 020	2 066 224	36.8	2.59	2.01	6.214	18.66	1.543	2.131	0.775	2.798
xz_1m_l4	1 076 057 785	23 230 841	5 156 845	16 486 476	1 587 520	22.2	2.01	3.06	6.832	24.25	1.730	4.355	0.697	2.351
xz_1m_l4e	2 198 787 666	35 410 889	13 089 895	20 222 586	2 098 408	37.0	2.43	1.84	6.173	17.52	1.620	2.109	0.800	2.919
imagick	117 733 556	2 444 490	2 450	2 437 082	4 958	0.1	0.02	4.14	4.422	81.36	10.069	69.568	1.302	0.348
blender	994 421 902	46 417 461	4 430 041	40 844 530	1 142 890	9.5	1.16	8.21	5.341	19.43	2.281	3.165	2.206	2.767
povray	2 240 986 547	168 867 726	2 121 674	155 851 654	10 894 398	1.3	0.27	13.91	10.145	41.21	7.406	20.677	1.842	1.807
omnetpp	12 426 267 350	842 780 888	5 233 033	791 025 406	46 522 449	0.6	0.09	12.73	12.565	26.59	0.250	0.863	1.933	2.478
xalancbmk	395 919 938	18 468 307	295 564	17 282 226	890 517	1.6	0.23	8.73	13.661	33.74	9.999	18.696	3.437	4.325
x264	475 535 613 470	19 842 562 937	13 777 288 367	5 322 257 634	743 016 936	69.4	6.24	2.24	2.326	26.97	0.389	0.954	1.487	1.610
gcc	16 026 850	1 021 294	52 739	952 288	16 267	5.2	0.84	11.88	7.167	32.52	0.666	2.169	1.438	1.209
mcf	24 426 982 047	234 360 896	91 410 910	142 147 076	802 910	39.0	1.58	1.16	25.499	75.88	5.567	8.282	3.676	5.825
perlbench_makerand	2 400 742 243	222 681 834	55	222 662 746	19 033	0.0	0.08	18.55	5.716	0.00	0.000	2.725	0.879	0.880
bwaves_1	17 288 358 363	145 483 800	145 451 376	26 481	5 943	100.0	3.55	0.00	41.067	31.22	2.704	0.330	4.063	5.189
bwaves_2	17 287 430 556	145 483 720	145 451 376	26 421	5 923	100.0	3.55	0.00	41.069	31.22	2.704	0.330	4.069	5.193
exchange2	57 667 143 431	1 874 880 445	1 304 771 747	345 493 349	224 615 349	69.6	5.03	1.20	2.504	1.08	2.742	3.560	7.229	4.833
fotonik3d	3 098 360 920	28 397 309	27 853 226	437 206	106 877	98.1	3.84	0.03	17.861	41.35	5.709	0.057	1.160	3.747
roms	28 955 766 090	203 007 943	182 777 748	12 238 510	7 991 685	90.0	4.79	0.08	20.998	38.82	4.438	1.564	1.413	2.838
wrf	84 396 898 728	420 514 578	109 794 207	285 402 467	25 317 904	26.1	0.91	0.68	18.055	22.28	9.398	6.610	0.844	1.636
cam4	33 466 803 539	585 237 550	168 288 974	404 426 599	12 521 977	28.8	2.11	2.42	10.243	52.99	2.656	2.913	1.165	1.702
Total	1 003 605 899 841	32 326 452 528	**21 110 519 840**	10 037 478 430	1 178 454 258	65.3	5.17	2.00	0.000	—	—	—	—	—

Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Register Spill Behaviour and Cache Hit/Miss Rates on RISC-V SPEC CPU2017

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Ismail Akturk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

4 November 2026

gem5 TimingSimpleCPU · workload profile `test` · L1D 4 KiB, L1I 32 KiB, L2 32 KiB, fully associative L1D/L2 (FALRU) · gem5 `e0a12aa4e3+dirty`

Definitions (metric schema 9). Every stack store issued inside the region of interest, and every later load that reads it back (the load's address lies in the bytes the store wrote; widths may differ), belongs to exactly one class. `spill`: register-allocator spill code, identified by GCC itself (`-dP` RTL dumps of the same build, memory operands in the LRA spill area). `csave`: callee-saved register save/restore, return address included, identified from the binary's CFI (`.eh_frame`). `other_stack`: every remaining SP/FP-relative stack store and reload (stack-passed arguments, memory-resident locals). `stack_*` is the sum of the three. For each class an *event* is one (store, first reload) pair, so `event` $\leq$ `store` and `load` $\geq$ `event`. All counters cover only the region delimited by `m5_work_begin/m5_work_end`: the gem5 statistics are dumped and reset at those boundaries, so the cache columns span the same window, and the cache tags carry the class, so per-class hit/miss rates come straight from gem5. *Spill %* and *Csave %* are the class's store+reload instructions over `ROI_insts`; *spill/stack %* is the share of stack events that are real spills.

Benchmark	ROI_insts	stack_event	**spill_event**	csave_event	other_event	spill/stack %	Spill %	Csave %	L1D m %	L2 m %	sp_load m %	sp_store m %	cs_load m %	cs_store m %
cactuBSSN	48 023 254 960	3 820 802 773	3 797 752 332	22 956 427	94 014	99.4	21.32	0.10	15.225	59.94	4.846	5.149	0.079	0.113
parest	37 262 401 281	130 109 038	16 148 624	110 058 554	3 901 860	12.4	0.22	0.59	9.718	68.60	1.354	1.455	0.193	0.415
deepsjeng	46 202 686 169	1 453 401 396	380 604 124	1 046 384 737	26 412 535	26.2	2.78	4.53	8.076	6.11	1.432	7.231	1.581	3.372
leela	26 063 891 644	808 702 134	26 356 444	749 189 404	33 156 286	3.3	0.36	5.75	5.074	27.89	0.327	0.422	0.940	0.358
namd	31 914 751 099	336 702 115	334 454 030	1 939 340	308 745	99.3	3.13	0.01	11.613	28.07	2.563	1.176	3.693	3.758
lbm	6 506 800 040	274 131 044	274 130 500	544	0	100.0	9.67	0.00	21.228	99.28	0.000	0.546	15.074	0.184
nab	6 691 483 051	152 456 167	124 535 928	24 348 560	3 571 679	81.7	7.90	0.73	3.889	8.31	0.131	0.108	0.619	0.726
xz	2 014 351 521	34 435 119	4 683 669	26 392 608	3 358 842	13.6	0.68	2.62	5.093	31.78	0.008	0.156	0.181	0.290
xz_4m_l1	2 001 462 938	30 569 523	3 959 703	23 548 312	3 061 508	13.0	0.59	2.35	6.622	41.40	0.009	0.191	0.420	0.300
xz_4m_l2	2 083 103 943	29 428 092	3 700 891	22 776 791	2 950 410	12.6	0.54	2.19	8.610	47.32	0.010	0.205	0.950	0.338
xz_4m_l3e	10 573 516 663	134 644 921	61 575 948	66 069 008	6 999 965	45.7	2.51	1.25	6.036	21.48	0.383	0.295	0.193	1.605
xz_4m_l4	4 011 967 585	68 464 242	20 213 033	44 411 548	3 839 661	29.5	2.12	2.21	6.387	38.80	0.085	0.645	0.496	0.190
xz_4m_l4e	12 115 711 352	137 799 962	63 036 496	67 532 844	7 230 622	45.7	2.28	1.11	5.901	19.54	0.508	0.316	0.198	1.884
xz_1m_l0	711 271 503	13 362 560	1 091 347	10 938 543	1 332 670	8.2	0.62	3.08	4.615	36.56	0.007	0.492	0.100	0.068
xz_1m_l1	719 848 327	12 739 426	973 549	10 458 508	1 307 369	7.6	0.58	2.91	5.203	42.80	0.007	0.541	0.162	0.105
xz_1m_l2	724 690 222	12 498 311	956 442	10 241 945	1 299 924	7.7	0.57	2.83	5.491	42.54	0.007	0.550	0.262	0.112
xz_1m_l3e	1 994 406 562	34 952 797	12 876 553	20 010 020	2 066 224	36.8	2.59	2.01	4.757	23.80	0.220	0.268	0.105	0.750
xz_1m_l4	1 076 057 785	23 230 841	5 156 845	16 486 476	1 587 520	22.2	2.01	3.06	4.642	34.83	0.017	0.603	0.245	0.089
xz_1m_l4e	2 198 787 666	35 410 889	13 089 895	20 222 586	2 098 408	37.0	2.43	1.84	4.806	21.91	0.319	0.279	0.103	0.892
imagick	117 733 190	2 444 490	2 450	2 437 082	4 958	0.1	0.02	4.14	4.156	87.38	7.201	58.930	0.284	0.302
blender	994 421 777	46 417 461	4 430 041	40 844 530	1 142 890	9.5	1.16	8.21	5.126	19.66	2.125	3.868	2.169	2.989
povray	2 240 986 446	168 867 726	2 121 674	155 851 654	10 894 398	1.3	0.27	13.91	8.700	50.57	7.064	19.522	1.658	0.953
omnetpp	12 426 246 371	842 780 719	5 233 033	791 025 233	46 522 453	0.6	0.09	12.73	11.530	27.53	0.054	0.774	1.152	2.269
xalancbmk	395 919 938	18 468 307	295 564	17 282 226	890 517	1.6	0.23	8.73	13.002	34.56	10.873	21.932	2.438	4.226
x264	475 535 613 470	19 842 562 937	13 777 288 367	5 322 257 634	743 016 936	69.4	6.24	2.24	1.836	31.44	0.155	0.852	0.929	1.181
gcc	16 026 850	1 021 294	52 739	952 288	16 267	5.2	0.84	11.88	5.686	38.30	0.223	1.341	0.409	0.635
mcf	24 426 982 047	234 360 896	91 410 910	142 147 076	802 910	39.0	1.58	1.16	24.518	78.48	3.115	6.856	2.578	5.257
perlbench_makerand	2 400 742 243	222 681 834	55	222 662 746	19 033	0.0	0.08	18.55	0.084	0.08	0.000	0.259	0.003	0.002
bwaves_1	17 288 358 363	145 483 800	145 451 376	26 481	5 943	100.0	3.55	0.00	7.312	95.60	0.859	0.001	3.384	5.019
bwaves_2	17 287 430 556	145 483 720	145 451 376	26 421	5 923	100.0	3.55	0.00	7.312	95.60	0.859	0.001	3.391	5.023
exchange2	57 667 143 431	1 874 880 445	1 304 771 747	345 493 349	224 615 349	69.6	5.03	1.20	1.538	1.79	1.694	2.611	3.452	2.322
fotonik3d	3 098 360 920	28 397 309	27 853 226	437 206	106 877	98.1	3.84	0.03	8.886	82.63	1.993	0.006	1.325	3.817
roms	28 955 766 090	203 007 943	182 777 748	12 238 510	7 991 685	90.0	4.79	0.08	10.763	64.31	2.165	0.970	0.303	2.174
wrf	84 396 891 536	420 514 371	109 794 207	285 402 322	25 317 842	26.1	0.91	0.68	5.883	61.76	5.434	4.485	0.483	1.248
cam4	33 466 803 502	585 237 544	168 288 974	404 426 594	12 521 976	28.8	2.11	2.42	8.802	59.48	1.461	2.470	0.630	1.409
Total	1 003 605 871 041	32 326 452 146	**21 110 519 840**	10 037 478 107	1 178 454 199	65.3	5.17	2.00	0.000	—	—	—	—	—

Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Register Spill Behaviour and Cache Hit/Miss Rates on RISC-V SPEC CPU2017

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Ismail Akturk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

4 November 2026

gem5 TimingSimpleCPU · workload profile `test` · L1D 4 KiB, L1I 32 KiB, L2 32 KiB, direct-mapped L1D/L2 · gem5 `e0a12aa4e3+dirty`

Definitions (metric schema 9). Every stack store issued inside the region of interest, and every later load that reads it back (the load's address lies in the bytes the store wrote; widths may differ), belongs to exactly one class. `spill`: register-allocator spill code, identified by GCC itself (`-dP` RTL dumps of the same build, memory operands in the LRA spill area). `csave`: callee-saved register save/restore, return address included, identified from the binary's CFI (`.eh_frame`). `other_stack`: every remaining SP/FP-relative stack store and reload (stack-passed arguments, memory-resident locals). `stack_*` is the sum of the three. For each class an *event* is one (store, first reload) pair, so `event` ≤ `store` and `load` ≥ `event`. All counters cover only the region delimited by `m5_work_begin/m5_work_end`: the gem5 statistics are dumped and reset at those boundaries, so the cache columns span the same window, and the cache tags carry the class, so per-class hit/miss rates come straight from gem5. *Spill %* and *Csave %* are the class's store+reload instructions over `ROI_insts`; *spill/stack %* is the share of stack events that are real spills.

Benchmark	ROI_insts	stack_event	**spill_event**	csave_event	other_event	spill/stack %	Spill %	Csave %	L1D m %	L2 m %	sp_load m %	sp_store m %	cs_load m %	cs_store m %
cactuBSSN	48 023 254 960	3 820 802 773	3 797 752 332	22 956 427	94 014	99.4	21.32	0.10	17.357	48.80	6.416	5.506	1.503	0.285
parest	37 262 401 281	130 109 038	16 148 624	110 058 554	3 901 860	12.4	0.22	0.59	15.359	47.58	20.704	6.626	4.432	5.347
deepsjeng	46 202 686 169	1 453 401 396	380 604 124	1 046 384 737	26 412 535	26.2	2.78	4.53	13.686	19.74	6.735	9.452	4.189	4.775
leela	26 063 891 644	808 702 134	26 356 444	749 189 404	33 156 286	3.3	0.36	5.75	8.956	17.75	19.596	0.455	3.522	4.806
namd	31 914 751 099	336 702 115	334 454 030	1 939 340	308 745	99.3	3.13	0.01	15.582	37.33	12.114	2.494	4.501	4.822
lbm	6 506 800 040	274 131 044	274 130 500	544	0	100.0	9.67	0.00	27.221	75.06	1.630	5.456	15.074	0.184
nab	6 691 483 051	152 456 167	124 535 928	24 348 560	3 571 679	81.7	7.90	0.73	10.786	15.30	1.540	1.648	2.584	2.927
xz	2 014 351 521	34 435 119	4 683 669	26 392 608	3 358 842	13.6	0.68	2.62	8.704	28.28	1.326	0.574	2.159	2.774
xz_4m_l1	2 001 462 938	30 569 523	3 959 703	23 548 312	3 061 508	13.0	0.59	2.35	10.079	36.30	1.450	0.593	2.726	1.808
xz_4m_l2	2 083 103 943	29 428 092	3 700 891	22 776 791	2 950 410	12.6	0.54	2.19	11.652	42.38	1.490	0.611	3.599	2.134
xz_4m_l3e	10 573 516 663	134 644 921	61 575 948	66 069 008	6 999 965	45.7	2.51	1.25	9.401	25.69	3.079	2.596	3.576	3.921
xz_4m_l4	4 011 967 585	68 464 242	20 213 033	44 411 548	3 839 661	29.5	2.12	2.21	10.814	33.53	3.070	3.152	3.148	2.674
xz_4m_l4e	12 115 711 352	137 799 962	63 036 496	67 532 844	7 230 622	45.7	2.28	1.11	9.165	24.56	3.194	2.620	3.659	4.145
xz_1m_l0	711 271 503	13 362 560	1 091 347	10 938 543	1 332 670	8.2	0.62	3.08	7.980	29.63	2.021	0.928	1.658	1.510
xz_1m_l1	719 848 327	12 739 426	973 549	10 458 508	1 307 369	7.6	0.58	2.91	8.793	33.78	2.138	0.935	1.717	1.579
xz_1m_l2	724 690 222	12 498 311	956 442	10 241 945	1 299 924	7.7	0.57	2.83	8.922	34.53	2.151	0.939	1.797	1.667
xz_1m_l3e	1 994 406 562	34 952 797	12 876 553	20 010 020	2 066 224	36.8	2.59	2.01	8.680	23.89	2.686	3.225	1.979	3.759
xz_1m_l4	1 076 057 785	23 230 841	5 156 845	16 486 476	1 587 520	22.2	2.01	3.06	8.845	28.51	2.900	4.939	1.754	3.271
xz_1m_l4e	2 198 787 666	35 410 889	13 089 895	20 222 586	2 098 408	37.0	2.43	1.84	8.884	22.40	2.786	3.234	2.007	3.876
imagick	117 733 190	2 444 490	2 450	2 437 082	4 958	0.1	0.02	4.14	7.434	47.92	16.947	70.350	13.154	0.436
blender	994 421 777	46 417 461	4 430 041	40 844 530	1 142 890	9.5	1.16	8.21	9.917	31.78	2.962	7.024	5.637	4.108
povray	2 241 026 793	168 869 468	2 121 674	155 853 333	10 894 461	1.3	0.27	13.91	13.662	37.82	13.927	21.862	3.772	4.330
omnetpp	12 426 246 371	842 780 719	5 233 033	791 025 233	46 522 453	0.6	0.09	12.73	15.055	36.64	0.724	10.668	3.306	3.556
xalancbmk	395 919 938	18 468 307	295 564	17 282 226	890 517	1.6	0.23	8.73	15.161	41.74	13.188	18.142	4.760	6.007
x264	475 535 613 470	19 842 562 937	13 777 288 367	5 322 257 634	743 016 936	69.4	6.24	2.24	3.981	26.05	0.981	1.203	1.895	2.419
gcc	16 026 850	1 021 294	52 739	952 288	16 267	5.2	0.84	11.88	10.137	32.44	2.274	4.977	3.935	2.807
mcf	24 426 982 047	234 360 896	91 410 910	142 147 076	802 910	39.0	1.58	1.16	28.220	72.37	8.955	10.166	5.465	6.451
perlbench_makerand	2 400 742 243	222 681 834	55	222 662 746	19 033	0.0	0.08	18.55	15.267	0.34	0.000	1.443	14.530	4.271
bwaves_1	17 288 358 363	145 483 800	145 451 376	26 481	5 943	100.0	3.55	0.00	44.118	27.18	5.350	1.371	4.864	6.053
bwaves_2	17 287 430 556	145 483 720	145 451 376	26 421	5 923	100.0	3.55	0.00	44.120	27.18	5.350	1.371	4.879	6.060
exchange2	57 667 143 431	1 874 880 445	1 304 771 747	345 493 349	224 615 349	69.6	5.03	1.20	4.008	1.04	3.416	3.666	4.345	3.792
fotonik3d	3 098 360 920	28 397 309	27 853 226	437 206	106 877	98.1	3.84	0.03	22.434	38.96	9.885	0.425	2.099	4.269
roms	28 955 766 090	203 007 943	182 777 748	12 238 510	7 991 685	90.0	4.79	0.08	30.012	38.90	7.109	3.376	3.203	4.921
wrf	84 396 927 626	420 516 285	109 794 207	285 403 859	25 318 219	26.1	0.91	0.68	26.370	26.50	13.692	9.129	4.905	2.812
cam4	33 466 803 465	585 237 538	168 288 974	404 426 589	12 521 975	28.8	2.11	2.42	14.018	44.30	5.427	4.410	2.154	2.157
Total	1 003 605 947 441	32 326 455 796	**21 110 519 840**	10 037 481 318	1 178 454 638	65.3	5.17	2.00	0.000	—	—	—	—	—

Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Register Spill Behaviour and Cache Hit/Miss Rates on RISC-V SPEC CPU2017

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Ismail Akturk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

4 November 2026

gem5 TimingSimpleCPU · workload profile `test` · L1D 4 KiB, L1I 32 KiB, L2 32 KiB, L1D 4-way, L2 8-way · gem5 `e0a12aa4e3+dirty`

Definitions (metric schema 9). Every stack store issued inside the region of interest, and every later load that reads it back (the load's address lies in the bytes the store wrote; widths may differ), belongs to exactly one class. `spill`: register-allocator spill code, identified by GCC itself (`-dP` RTL dumps of the same build, memory operands in the LRA spill area). `csave`: callee-saved register save/restore, return address included, identified from the binary's CFI (`.eh_frame`). `other_stack`: every remaining SP/FP-relative stack store and reload (stack-passed arguments, memory-resident locals). `stack_*` is the sum of the three. For each class an *event* is one (store, first reload) pair, so `event` ≤ `store` and `load` ≥ `event`. All counters cover only the region delimited by `m5_work_begin/m5_work_end`: the gem5 statistics are dumped and reset at those boundaries, so the cache columns span the same window, and the cache tags carry the class, so per-class hit/miss rates come straight from gem5. *Spill %* and *Csave %* are the class's store+reload instructions over `ROI_insts`; *spill/stack %* is the share of stack events that are real spills.

Benchmark	ROI_insts	stack_event	**spill_event**	csave_event	other_event	spill/stack %	Spill %	Csave %	L1D m %	L2 m %	sp_load m %	sp_store m %	cs_load m %	cs_store m %
cactuBSSN	48 023 254 960	3 820 802 773	3 797 752 332	22 956 427	94 014	99.4	21.32	0.10	15.296	65.69	4.855	4.272	0.084	0.119
parest	37 262 401 281	130 109 038	16 148 624	110 058 554	3 901 860	12.4	0.22	0.59	10.210	65.62	2.848	1.609	0.422	0.901
deepsjeng	46 202 686 169	1 453 401 396	380 604 124	1 046 384 737	26 412 535	26.2	2.78	4.53	8.904	7.54	2.010	7.255	2.447	3.442
leela	26 063 891 644	808 702 134	26 356 444	749 189 404	33 156 286	3.3	0.36	5.75	5.854	24.20	4.140	0.422	1.494	0.454
namd	31 914 751 099	336 702 115	334 454 030	1 939 340	308 745	99.3	3.13	0.01	11.705	28.28	2.779	1.143	3.769	3.776
lbm	6 506 800 040	274 131 044	274 130 500	544	0	100.0	9.67	0.00	21.660	97.36	0.000	0.675	15.074	0.184
nab	6 691 483 051	152 456 167	124 535 928	24 348 560	3 571 679	81.7	7.90	0.73	5.327	6.02	0.506	0.311	0.637	0.759
xz	2 014 351 521	34 435 119	4 683 669	26 392 608	3 358 842	13.6	0.68	2.62	5.694	29.53	0.008	0.207	1.591	1.545
xz_4m_l1	2 001 462 938	30 569 523	3 959 703	23 548 312	3 061 508	13.0	0.59	2.35	7.171	39.21	0.009	0.223	2.255	1.182
xz_4m_l2	2 083 103 943	29 428 092	3 700 891	22 776 791	2 950 410	12.6	0.54	2.19	9.127	45.18	0.010	0.240	3.054	1.179
xz_4m_l3e	10 573 516 663	134 644 921	61 575 948	66 069 008	6 999 965	45.7	2.51	1.25	6.348	20.93	1.036	0.505	0.782	1.822
xz_4m_l4	4 011 967 585	68 464 242	20 213 033	44 411 548	3 839 661	29.5	2.12	2.21	7.394	34.26	0.983	0.992	0.812	0.663
xz_4m_l4e	12 115 711 352	137 799 962	63 036 496	67 532 844	7 230 622	45.7	2.28	1.11	6.199	19.17	1.151	0.518	0.827	2.083
xz_1m_l0	711 271 503	13 362 560	1 091 347	10 938 543	1 332 670	8.2	0.62	3.08	5.145	33.71	0.007	0.702	0.710	0.671
xz_1m_l1	719 848 327	12 739 426	973 549	10 458 508	1 307 369	7.6	0.58	2.91	5.814	39.07	0.007	0.667	0.890	0.685
xz_1m_l2	724 690 222	12 498 311	956 442	10 241 945	1 299 924	7.7	0.57	2.83	6.067	39.07	0.007	0.681	0.961	0.694
xz_1m_l3e	1 994 406 562	34 952 797	12 876 553	20 010 020	2 066 224	36.8	2.59	2.01	5.462	21.23	0.742	0.675	0.307	1.086
xz_1m_l4	1 076 057 785	23 230 841	5 156 845	16 486 476	1 587 520	22.2	2.01	3.06	5.828	28.43	0.697	1.355	0.357	0.534
xz_1m_l4e	2 198 787 666	35 410 889	13 089 895	20 222 586	2 098 408	37.0	2.43	1.84	5.473	19.76	0.830	0.678	0.318	1.220
imagick	117 733 556	2 444 490	2 450	2 437 082	4 958	0.1	0.02	4.14	4.200	85.57	8.465	68.436	0.324	0.318
blender	994 421 902	46 417 461	4 430 041	40 844 530	1 142 890	9.5	1.16	8.21	5.064	20.38	3.273	3.869	1.857	2.825
povray	2 240 986 446	168 867 726	2 121 674	155 851 654	10 894 398	1.3	0.27	13.91	9.134	45.58	6.987	19.611	1.736	1.170
omnetpp	12 426 267 350	842 780 888	5 233 033	791 025 406	46 522 449	0.6	0.09	12.73	11.851	28.01	0.054	0.821	1.410	2.079
xalancbmk	395 919 938	18 468 307	295 564	17 282 226	890 517	1.6	0.23	8.73	13.280	34.60	9.920	18.315	2.541	4.039
x264	475 535 613 470	19 842 562 937	13 777 288 367	5 322 257 634	743 016 936	69.4	6.24	2.24	1.966	31.00	0.220	0.886	0.994	1.293
gcc	16 026 850	1 021 294	52 739	952 288	16 267	5.2	0.84	11.88	6.105	36.94	0.333	1.477	0.616	0.817
mcf	24 426 982 047	234 360 896	91 410 910	142 147 076	802 910	39.0	1.58	1.16	24.847	77.84	4.001	7.208	2.778	5.440
perlbench_makerand	2 400 742 243	222 681 834	55	222 662 746	19 033	0.0	0.08	18.55	3.756	0.00	0.000	6.661	0.003	0.002
bwaves_1	17 288 358 363	145 483 800	145 451 376	26 481	5 943	100.0	3.55	0.00	40.946	31.23	3.813	0.021	3.312	4.917
bwaves_2	17 287 430 556	145 483 720	145 451 376	26 421	5 923	100.0	3.55	0.00	40.947	31.23	3.813	0.021	3.316	4.920
exchange2	57 667 143 431	1 874 880 445	1 304 771 747	345 493 349	224 615 349	69.6	5.03	1.20	2.008	1.35	1.744	2.983	4.998	2.280
fotonik3d	3 098 360 920	28 397 309	27 853 226	437 206	106 877	98.1	3.84	0.03	11.962	61.79	3.816	0.008	0.860	3.556
roms	28 955 766 090	203 007 943	182 777 748	12 238 510	7 991 685	90.0	4.79	0.08	14.936	53.16	3.051	0.793	0.316	2.387
wrf	84 396 893 159	420 514 417	109 794 207	285 402 352	25 317 858	26.1	0.91	0.68	11.550	34.19	6.880	4.952	0.529	1.411
cam4	33 466 803 502	585 237 544	168 288 974	404 426 594	12 521 976	28.8	2.11	2.42	9.074	59.49	1.792	2.696	0.683	1.471
Total	1 003 605 894 134	32 326 452 361	**21 110 519 840**	10 037 478 310	1 178 454 211	65.3	5.17	2.00	0.000	—	—	—	—	—

Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Register Spill Behaviour and Cache Hit/Miss Rates on RISC-V SPEC CPU2017

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Ismail Akturk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

4 November 2026

gem5 TimingSimpleCPU · workload profile `test` · L1D 4 KiB, L1I 32 KiB, L2 32 KiB, L1D 8-way, L2 8-way · gem5 `e0a12aa4e3+dirty`

Definitions (metric schema 9). Every stack store issued inside the region of interest, and every later load that reads it back (the load's address lies in the bytes the store wrote; widths may differ), belongs to exactly one class. `spill`: register-allocator spill code, identified by GCC itself (`-dP` RTL dumps of the same build, memory operands in the LRA spill area). `csave`: callee-saved register save/restore, return address included, identified from the binary's CFI (`.eh_frame`). `other_stack`: every remaining SP/FP-relative stack store and reload (stack-passed arguments, memory-resident locals). `stack_*` is the sum of the three. For each class an *event* is one (store, first reload) pair, so `event` ≤ `store` and `load` ≥ `event`. All counters cover only the region delimited by `m5_work_begin/m5_work_end`: the gem5 statistics are dumped and reset at those boundaries, so the cache columns span the same window, and the cache tags carry the class, so per-class hit/miss rates come straight from gem5. *Spill %* and *Csave %* are the class's store+reload instructions over `ROI_insts`; *spill/stack %* is the share of stack events that are real spills.

Benchmark	ROI_insts	stack_event	**spill_event**	csave_event	other_event	spill/stack %	Spill %	Csave %	L1D m %	L2 m %	sp_load m %	sp_store m %	cs_load m %	cs_store m %
cactuBSSN	48 023 254 960	3 820 802 773	3 797 752 332	22 956 427	94 014	99.4	21.32	0.10	15.623	64.56	5.020	4.669	0.082	0.119
parest	37 262 401 281	130 109 038	16 148 624	110 058 554	3 901 860	12.4	0.22	0.59	9.895	67.70	1.416	1.473	0.225	0.627
deepsjeng	46 202 686 169	1 453 401 396	380 604 124	1 046 384 737	26 412 535	26.2	2.78	4.53	8.627	7.70	1.774	7.102	2.089	3.483
leela	26 063 891 644	808 702 134	26 356 444	749 189 404	33 156 286	3.3	0.36	5.75	5.161	27.19	0.591	0.422	1.011	0.368
namd	31 914 751 099	336 702 115	334 454 030	1 939 340	308 745	99.3	3.13	0.01	11.605	28.50	2.551	1.166	3.711	3.765
lbm	6 506 800 040	274 131 044	274 130 500	544	0	100.0	9.67	0.00	21.730	97.05	0.000	0.633	15.074	0.184
nab	6 691 483 051	152 456 167	124 535 928	24 348 560	3 571 679	81.7	7.90	0.73	4.074	7.87	0.156	0.129	0.628	0.732
xz	2 014 351 521	34 435 119	4 683 669	26 392 608	3 358 842	13.6	0.68	2.62	5.341	31.37	0.008	0.135	0.455	0.617
xz_4m_l1	2 001 462 938	30 569 523	3 959 703	23 548 312	3 061 508	13.0	0.59	2.35	6.788	41.37	0.009	0.167	0.994	0.605
xz_4m_l2	2 083 103 943	29 428 092	3 700 891	22 776 791	2 950 410	12.6	0.54	2.19	8.809	46.80	0.010	0.178	2.158	0.572
xz_4m_l3e	10 573 516 663	134 644 921	61 575 948	66 069 008	6 999 965	45.7	2.51	1.25	6.094	21.75	0.449	0.362	0.371	1.391
xz_4m_l4	4 011 967 585	68 464 242	20 213 033	44 411 548	3 839 661	29.5	2.12	2.21	6.791	37.27	0.230	0.751	0.552	0.305
xz_4m_l4e	12 115 711 352	137 799 962	63 036 496	67 532 844	7 230 622	45.7	2.28	1.11	5.967	19.86	0.573	0.377	0.399	1.663
xz_1m_l0	711 271 503	13 362 560	1 091 347	10 938 543	1 332 670	8.2	0.62	3.08	4.777	36.16	0.007	0.402	0.180	0.135
xz_1m_l1	719 848 327	12 739 426	973 549	10 458 508	1 307 369	7.6	0.58	2.91	5.395	42.00	0.007	0.444	0.308	0.188
xz_1m_l2	724 690 222	12 498 311	956 442	10 241 945	1 299 924	7.7	0.57	2.83	5.685	41.63	0.007	0.450	0.524	0.185
xz_1m_l3e	1 994 406 562	34 952 797	12 876 553	20 010 020	2 066 224	36.8	2.59	2.01	4.970	23.27	0.271	0.382	0.148	0.706
xz_1m_l4	1 076 057 785	23 230 841	5 156 845	16 486 476	1 587 520	22.2	2.01	3.06	5.046	32.77	0.090	0.779	0.249	0.180
xz_1m_l4e	2 198 787 666	35 410 889	13 089 895	20 222 586	2 098 408	37.0	2.43	1.84	5.016	21.50	0.367	0.391	0.154	0.842
imagick	117 733 556	2 444 490	2 450	2 437 082	4 958	0.1	0.02	4.14	4.168	86.22	7.194	58.765	0.298	0.307
blender	994 421 902	46 417 461	4 430 041	40 844 530	1 142 890	9.5	1.16	8.21	5.037	20.42	2.141	3.868	2.043	2.995
povray	2 240 986 446	168 867 726	2 121 674	155 851 654	10 894 398	1.3	0.27	13.91	8.697	47.73	6.932	19.449	1.477	1.013
omnetpp	12 426 267 350	842 780 888	5 233 033	791 025 406	46 522 449	0.6	0.09	12.73	11.572	28.59	0.053	0.721	1.230	2.192
xalancbmk	395 919 938	18 468 307	295 564	17 282 226	890 517	1.6	0.23	8.73	13.117	35.01	10.128	18.898	2.513	3.924
x264	475 535 613 470	19 842 562 937	13 777 288 367	5 322 257 634	743 016 936	69.4	6.24	2.24	1.858	32.44	0.158	0.861	0.937	1.233
gcc	16 026 850	1 021 294	52 739	952 288	16 267	5.2	0.84	11.88	5.733	38.80	0.234	1.328	0.443	0.672
mcf	24 426 982 047	234 360 896	91 410 910	142 147 076	802 910	39.0	1.58	1.16	24.643	78.48	3.545	6.915	2.675	5.246
perlbench_makerand	2 400 742 243	222 681 834	55	222 662 746	19 033	0.0	0.08	18.55	3.795	0.00	0.000	99.996	0.003	0.002
bwaves_1	17 288 358 363	145 483 800	145 451 376	26 481	5 943	100.0	3.55	0.00	26.018	37.38	5.467	0.021	3.308	4.973
bwaves_2	17 287 430 556	145 483 720	145 451 376	26 421	5 923	100.0	3.55	0.00	26.019	37.38	5.467	0.021	3.304	4.977
exchange2	57 667 143 431	1 874 880 445	1 304 771 747	345 493 349	224 615 349	69.6	5.03	1.20	1.342	2.01	1.430	2.700	3.470	2.262
fotonik3d	3 098 360 920	28 397 309	27 853 226	437 206	106 877	98.1	3.84	0.03	9.207	80.25	2.496	0.006	1.060	3.788
roms	28 955 766 090	203 007 943	182 777 748	12 238 510	7 991 685	90.0	4.79	0.08	11.833	67.05	2.452	0.885	0.302	2.456
wrf	84 396 891 740	420 514 378	109 794 207	285 402 322	25 317 849	26.1	0.91	0.68	6.920	55.78	6.233	4.759	0.462	1.292
cam4	33 466 803 539	585 237 550	168 288 974	404 426 599	12 521 977	28.8	2.11	2.42	8.840	60.99	1.527	2.519	0.613	1.469
Total	1 003 605 892 752	32 326 452 328	**21 110 519 840**	10 037 478 285	1 178 454 203	65.3	5.17	2.00	0.000	—	—	—	—	—